

Controle com argumentos

Podemos ajustar `begin` e `end` de `LinearGradient`:

```
gradient: LinearGradient(  
  colors: [  
    Colors.blue,  
    Colors.lightBlue,  
  ],  
  begin: Alignment.topCenter,  
  end: Alignment.bottomCenter,  
),
```

Prática

- Ajuste o tamanho do texto para 32 e a cor para destacar.

<https://api.flutter.dev/flutter/painting/TextStyle-class.html>

```
child: Text(  
  'Hello, world!',  
  style: TextStyle(  
    color: Colors.white,  
    fontSize: 28,  
  ),  
),
```

Separando widgets

Se identificar um Widget que vai usar no futuro em outras aplicações, você pode extrair ele do código:

```
class CaixaTexto extends StatelessWidget {  
  const CaixaTexto({  
    super.key,  
  });  
  
  @override  
  Widget build(BuildContext context) {  
    return Container(  
      decoration: const BoxDecoration(  

```

Separando widgets

```
const MaterialApp(  
  home: Scaffold(  
    //backgroundColor: Colors.lightBlue,  
    body: CaixaTexto(),  
  ),  
),
```

Criando biblioteca

Podemos colocar o Widget em um arquivo separado `lib/utis.dart` e continuar usando.

```
import 'package:flutter/material.dart';  
import 'package:flutter_application_1/utils.dart';  
  
void main() {  
  runApp(  
    const MaterialApp(  
      home: Scaffold(  
        //backgroundColor: Colors.lightBlue,  
        body: CaixaTexto(),  
      ),  
    ),  
  );  
}
```

Prática

Crie um widget separado para o Text. Nome: TextoComEstilo.


```
class TextoComEstilo extends StatelessWidget {  
  const TextoComEstilo({  
    super.key,  
  });  
  
  @override  
  Widget build(BuildContext context) {  
    return Text(  
      'Hello, world!',  
    );  
  }  
}
```

Usando variáveis

```
var tamanhoFonte = 28.0;
var corFonte = Colors.white;

class TextoComEstilo extends StatelessWidget {
  //...
  style: TextStyle(
    color: corFonte,
    fontSize: tamanhoFonte,
  ),
  //...
```

Ajuste de tipos

```
double tamanhoFonte = 28.0;  
Color corFonte = Colors.white;
```

Atribuição

```
double? tamanhoFonte;  
Color? corFonte;  
  
class TextoComEstilo extends StatelessWidget {  
  //...  
  
  @override  
  Widget build(BuildContext context) {  
    tamanhoFonte = 28.0;  
    corFonte = Colors.white;  
    return Text(  
  //...
```

const

- const: Marcado como const o valor não muda mais. O valor deve ser conhecido na compilação.

```
const tamanhoFonte = 28.0;
```

final

- final: Marcado como final o valor não muda, mas pode ser calculado na execução.

```
double calcula(double x) {  
    return x * 2;  
}  
  
final tamanhoFonte = calcula(14.0);
```

Variáveis de instância

- Podemos construir Widgets ajustáveis com variáveis.

```
class TextoComEstilo extends StatelessWidget {  
  TextoComEstilo(  
    String texto, {  
    super.key,  
  }) : textoMostrado = texto;  
  
  String textoMostrado;  
  
  @override  
  Widget build(BuildContext context) {  
    return Text(textoMostrado);  
  }  
}
```



```
return MaterialApp(  
  home: Scaffold(  
    body: Center(  
      child: TextoComEstilo('Hello world!'),  
    ),  
  ),  
);
```

Podemos ajustar a propriedade na chamada

```
class TextoComEstilo extends StatelessWidget {  
  TextoComEstilo(  
    this.textoMostrado, {  
    super.key,  
  });  
  
  String textoMostrado; // pode usar final aqui para indicar que não muda.  
  
  @override  
  Widget build(BuildContext context) {  
    return Text(textoMostrado);  
  }  
}
```

Prática

Ajuste o TextoComEstilo para receber também o tamanho da fonte.

```
child: TextoComEstilo('Hello world!', tamanho: 28.0),
```